

Assignment 2 (DFA & NFA)

Instructions

- The assignment is submitted in groups of **minimum 3 and maximum 4** students from the **same lab** or the **same TA**.
- The Deadline for submission is on **29/4/2025 at 11:59**.
- Submission will be on Google Classroom. No late submission, or through e-mail submission is allowed.
- Please submit one compressed folder with the **.java files and pdf file**. The folder name should follow this structure: ID1_ID2_ID3_ID4_GROUP.zip
- In case of Cheating, you will get a negative grade whether you give the code to someone, take the code from someone/internet, or even send it to someone for any reason.

Requirements

- Solve all the 10 problems.
- For each problem, you should submit the **drawn FSA** and **Java code** that solves it.
- For all problems write in **Java Language**.
- Submit a **pdf file** that contains the drawn DFA or NFA for the problems.
- The Java program will take **ONE** input text file containing the input values for each problem, the program must save the output of each problem in **ONE** output text file following the required format for each problem.

Input file:

Starts with the number of the problem, followed by the input values for the problem, then “end” to indicate the input for this problem is finished and to move to the next problem.

Output file:

Starts with the number of the problem, followed by the corresponding output values of the input for the problem, then “x” to indicate the output for this problem is finished and to move to the next problem.

Important Note:

All the problems may be tested with **different test cases**, so ensure to construct and Simulate FSA that cover and handle all edge cases.

DFA

Simulate a Deterministic Finite Automaton:

- The DFA's states
- The alphabet
- The transition function
- The start state
- The set of accepting states

You must simulate a Deterministic Finite Automaton (DFA) by implementing the following components and ensuring correct DFA behavior:

States: Define the set of states in the DFA.

Alphabet: Specify the input symbols the DFA recognizes

Transition Function: Map each state-symbol pair to a next state.

Start State: Identify the initial state of the DFA

Accepting States: Define the set of states that indicate acceptance.

For each problem, **pass the DFA configuration** (States, Alphabet, Transition, Start State, Accepting States) to your simulation class(or function) and ensure it correctly processes input strings.

Code Example:

You may use this class to simulate DFA (you can add any methods or properties you need , but **you are not allowed** to delete the below methods)

```
public class DFA
{
    public DFA(int[] states,int startState,int[] finalState,char[]
alphabet,Map<Integer, int[]> transitionTable)
    {

    }

    public boolean isAccepted(String s)
    {
        // return true if string valid
        // return false if string invalid
    }

    public void solveProblem(BufferedReader br, BufferedWriter bw)
    {
        // read input from input file then call isAccept method
        // and write result in output file
    }
}
```

for each problem you may create class that contain FSA information then use this class in main for Ex:

```
public class Problem1
{
    int[] states = {0, 1, 2, 3};
    int startState = 0;
    int[] finalState = {1, 2};
    char[] alphabet = {'a', 'b'};
    public Map<Integer, int[]> transitionTable = Map.ofEntries(
        entry(0, new int[]{1, 2}),
        entry(1, new int[]{1, 2}),
        entry(2, new int[]{3, 2}),
        entry(3, new int[]{3, 3})
    );
    DFA dfa = new DFA(states,startState,finalState,alphabet,transitionTable);
    public Problem1 (BufferedReader br, BufferedWriter bw)
    {
        dfa.solveProblem(br,bw);
    }
}
```

Problem 1

Design a DFA that accepts all strings which do not contain the even number of a over $\{a, b\}$.

Input	Output
1	1
ab	True
ba	True
aba	False
bbb	False
end	x

Problem 2

Design a **DFA** that accepts all strings over $\{0,1\}$ that start with **1** and have a length of at least **3**.

Input	Output
2	2
1	False
10	False
100	True
0111	False
end	x

Problem 3

Design a **DFA** that accepts all strings over $\{x, y, z\}$ that contain the substring "xyz".

Input	Output
3	3
xyxz	False
zyxyx	False
yzxyz	True
yxzyyxyzx	True
end	x

Problem 4

Design a **DFA** that accepts all binary strings that represent numbers **divisible by 7**.

Input	Output
4	4
0	True
111	True
1100011	False
1010101	False
end	x

Problem 5

Design a **DFA** that accepts all strings over **{a, b}** where every 'a' appears in a consecutive pair.

Input	Output
5	5
aab	True
baabbbbbaaab	True
baaaaaabaa	True
baaabbaa	False
end	x

Problem 6

Design a **DFA** that accepts all strings that contain an **even number of 'a'** and **odd number of b** over the alphabet **{a, b}**

Input	Output
6	6
aab	True
babba	True
b	True
baab	False
end	x

NFA

Simulate a Nondeterministic Finite Automaton (NFA):

- The NFA's states
- The alphabet
- The transition function.
- The start state(s)
- The set of accepting states

You must simulate a Nondeterministic Finite Automaton (NFA) by implementing the following components and ensuring correct NFA behavior:

States: Define the set of states in the NFA.

Alphabet: Specify the input symbols the NFA recognizes

Transition Function: Map each state-symbol pair to a next state.

Start State: Identify the initial state of the NFA

Accepting States: Define the set of states that indicate acceptance.

For each problem, pass the NFA configuration (States, Alphabet, Transition, Start State, Accepting States) to your simulation class (or function) and ensure it correctly processes input strings.

Code Example:

You may Use this class to simulate NFA (you can add any methods or properties you need , but **you are not allowed** to delete the below methods)

```
public class NFA
{
    public NFA(int[] states, int[] startState, int[] finalState, char[]
alphabet, Map<Integer, int[][]> transitionTable)
    {

    }

    public boolean isAccepted(String input)
    {
        // return true if string valid
        // return false if string invalid
    }

    public void solveProblem(BufferedReader br, BufferedWriter bw)
    {
        // read input from input file then call isAccept method
        // and write result in output file
    }
}
```

for each problem you may create class that contain FSA information then use this class in main for Ex:

```
public class Problem7
{
    int[] states = {0, 1, 2, 3,4, 5};
    int startState[] = {0};
    int[] finalState = {5};
    char[] alphabet = {'E','0', '1'};
    public Map<Integer, int[][]> transitionTable = Map.ofEntries(
        entry(0, new int[][]{{-1}, {0,2}, {0,1}}),
        entry(1, new int[][]{{-1}, {3}, {-1}}),
        entry(2, new int[][]{{-1}, {-1},{4}}),
        entry(3, new int[][]{{-1}, {-1}, {5}}),
        entry(4, new int[][]{{-1}, {5}, {-1}}),
        entry(5,new int[][]{{-1}, {5}, {5}})
    );

    NFA nfa = new NFA(states,startState,finalState,alphabet,transitionTable);

    public Problem7 (BufferedReader br, BufferedWriter bw)
    {
        nfa.solveProblem(br,bw);
    }
}
```

Problem 7

Design an **NFA** that accepts all binary strings containing **at least one '0' and at least one 1** and Do not have more than two consecutive identical digits. ('111','000' not allowed)

Input

7

1001

0101100

10011110110

11

end

Output

7

True

True

False

False

x

Problem 8

Design an **NFA** that accepts all strings over **{a, b}** where the substring **"ab"** appears at least twice.

Input

8

abaaab

aaabbaba

abaaaba

bbbabbb

end

Output

8

True

True

True

False

x

Problem 9

Design an **NFA** that accepts all binary strings where **every '1' is followed by at most one '0'**.

Input	Output
9	9
10111	True
11111	True
010101010101	True
0100	False
end	x

Problem 10

Design an **NFA** that accepts all strings over **{x, y}** where **no two 'y' appear consecutively**.

Input	Output
10	10
xyxxxxyx	True
xxxxxxxxxx	True
xyxxxxyxyxyx	True
xyxyyxyxy	False
end	x